

Name :- Mihir Mungara

Snowflake Assignment-13 : Implement a Real-Time Analytics Pipeline Using Streams and Tasks

Snowflake Real-Time Analytics Pipeline and Time Travel Demonstration

Objective

This exercise demonstrates:

- 1. Real-Time Analytics using Streams and Tasks.
- 2. Historical Data Recovery and Analysis using Time Travel.
- 3. Dashboard Integration using Power BI or Tableau.

Part 1: Real-Time Analytics Pipeline Using Streams and Tasks

Architecture

Source Table --> Stream --> Task --> Target Analytics Table --> Power BI

Step 1: Create Source Table

```
CREATE OR REPLACE TABLE SALES_SOURCE (  
  SALE_ID INT,  
  PRODUCT_NAME STRING,  
  QUANTITY INT,  
  SALE_AMOUNT NUMBER(10,2),  
  SALE_DATE TIMESTAMP  
);
```

Insert Initial Data:

```
INSERT INTO SALES_SOURCE VALUES  
(1, 'Laptop', 2, 100000, CURRENT_TIMESTAMP()),  
(2, 'Mobile', 1, 25000, CURRENT_TIMESTAMP());
```

```
SELECT * FROM SALES_SOURCE;
```

	# SALE_ID	A PRODUCT_NAME	# QUANTITY	# SALE_AMOUNT	NT SALE_DATE
1	1	Laptop	2	100000.00	2026-06-05 02:01:40.567
2	2	Mobile	1	25000.00	2026-06-05 02:01:40.567

Step 2: Create Target Analytics Table

```
CREATE OR REPLACE TABLE SALES_ANALYTICS (  
  SALE_ID INT,  
  PRODUCT_NAME STRING,  
  QUANTITY INT,  
  SALE_AMOUNT NUMBER(10,2),  
  SALE_DATE TIMESTAMP  
);
```

Step 3: Create Stream

The stream captures INSERT, UPDATE, and DELETE operations.

```
CREATE OR REPLACE STREAM SALES_STREAM  
ON TABLE SALES_SOURCE;
```

Verify:

```
SHOW STREAMS;
```

Step 4: Generate New Data

```
INSERT INTO SALES_SOURCE VALUES  
(3, 'Tablet', 1, 15000, CURRENT_TIMESTAMP()),  
(4, 'Monitor', 2, 30000, CURRENT_TIMESTAMP());
```

Check stream contents:

```
SELECT * FROM SALES_STREAM;
```

Expected:

```
METADATA$ACTION = INSERT
```

	# SALE_ID	△ PRODUCT_NAME	# QUANTITY	# SALE_AMOUNT	🕒 SALE_DATE	⏮ ⏭	△ METADATA\$ACTION	0/1 METADATA\$ISUPDATE	△ METADATA\$ROW_ID
1	3	Tablet	1	15000.00	2026-06-05 02:03:31.704		INSERT	FALSE	baba0a2ce3a041854e5f248d42da129da7dc9c08
2	4	Monitor	2	30000.00	2026-06-05 02:03:31.704		INSERT	FALSE	8d2aecebe504809298a905ce7d0e564fbeb5bfa9

Step 5: Create Task

The task processes new records every minute.

```
CREATE OR REPLACE TASK SALES_TASK
WAREHOUSE = COMPUTE_WH
SCHEDULE = '1 MINUTE'
AS
INSERT INTO SALES_ANALYTICS
SELECT
    SALE_ID,
    PRODUCT_NAME,
    QUANTITY,
    SALE_AMOUNT,
    SALE_DATE
FROM SALES_STREAM
WHERE METADATA$ACTION = 'INSERT';
```

Step 6: Enable Task

```
ALTER TASK SALES_TASK RESUME;
```

Verify:

```
SHOW TASKS;
```

Step 7: Monitor Task Execution

```
SELECT *
FROM TABLE(
    INFORMATION_SCHEMA.TASK_HISTORY()
)
ORDER BY SCHEDULED_TIME DESC;
```

Step 8: Verify Analytics Table

After task execution:

```
SELECT * FROM SALES_ANALYTICS;
```

Expected:

ID	# SALE_ID	PRODUCT_NAME	# QUANTITY	# SALE_AMOUNT	DATE SALE_DATE
1	3	Tablet	1	15000.00	2026-06-05 02:03:31.704
2	4	Monitor	2	30000.00	2026-06-05 02:03:31.704

Dashboard Integration

Power BI

Connect Snowflake

1. Open Power BI Desktop.
2. Get Data → Snowflake.
3. Enter:
 - a. Server
 - b. Warehouse
 - c. Database
 - d. Schema
4. Select: SALES_ANALYTICS
5. Load Data.

Create Dashboard

Visuals:

KPI Card

Total Sales

Bar Chart

Product Name vs Sale Amount

Line Chart

Sale Date vs Sale Amount

Whenever new records arrive:

SALES_SOURCE



SALES_STREAM



SALES_TASK

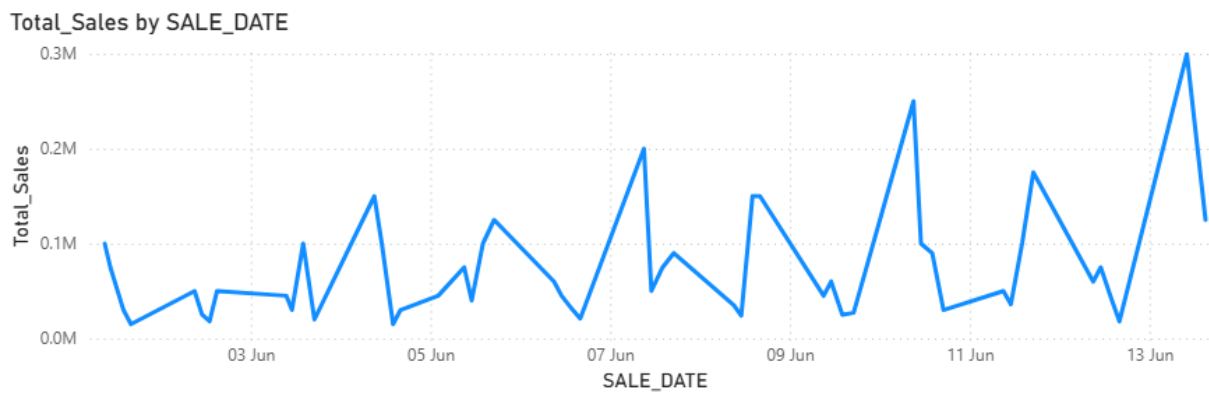
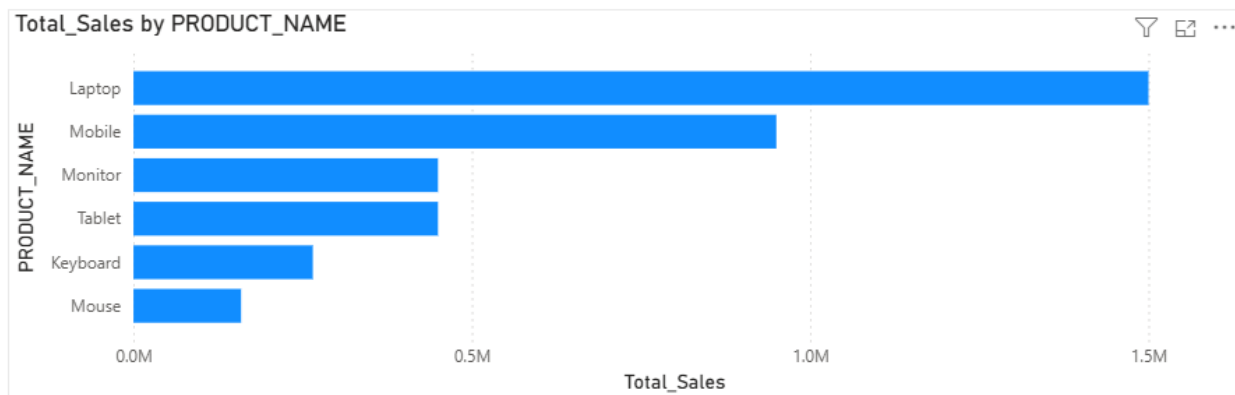


SALES_ANALYTICS



Dashboard Refresh

Dashboard updates automatically.



Real-Time Analytics Pipeline Documentation

Components

Component	Purpose
SALES_SOURCE	Incoming transactional data
SALES_STREAM	Captures changes
SALES_TASK	Processes changes automatically
SALES_ANALYTICS	Reporting table
Power BI/Tableau	Visualization layer

Benefits

- Near real-time reporting.
- Automated ETL process.
- Minimal manual intervention.
- Efficient incremental processing.

Part 2: Demonstrate Time Travel

Objective

Use Time Travel to:

- Query historical data.
- Recover deleted data.
- Compare table versions.

Step 1: Create Sample Table

```
CREATE OR REPLACE TABLE EMPLOYEES (  
    EMP_ID INT,  
    EMP_NAME STRING,  
    SALARY NUMBER  
);
```

Insert data:

```
INSERT INTO EMPLOYEES VALUES  
(1, 'Alice', 50000),  
(2, 'Bob', 60000),  
(3, 'Charlie', 70000);
```


Verify:

```
SELECT * FROM EMPLOYEES;
```

	EMP_ID	EMP_NAME	SALARY
1	1	Alice	50000
2	2	Bob	60000
3	3	Charlie	70000

Step 2: Capture Current Timestamp

```
SELECT CURRENT_TIMESTAMP();
```

Example:

2026-06-05 03:42:32.612 -0700

Save this timestamp.

Step 3: Delete Data

```
DELETE FROM EMPLOYEES  
WHERE EMP_ID = 2;
```

Verify:

```
SELECT * FROM EMPLOYEES;
```

Output:

	EMP_ID	EMP_NAME	SALARY
1	1	Alice	50000
2	3	Charlie	70000

Step 4: Query Historical Data

Retrieve table state before deletion:

```
SELECT *  
FROM EMPLOYEES  
AT (  
    TIMESTAMP => '2026-06-05 03:42:32.612 -0700'  
);
```

Output:

EMP_ID	EMP_NAME	SALARY
1	Alice	50000
2	Bob	60000
3	Charlie	70000

Step 5: Compare Data States

Current Data:

```
SELECT * FROM EMPLOYEES;
```

Historical Data:

```
SELECT *  
FROM EMPLOYEES  
AT (  
    TIMESTAMP => '2026-06-05 03:42:32.612 -0700'  
);
```

Comparison shows deleted records.

Step 6: Recover Deleted Rows

```
INSERT INTO EMPLOYEES  
SELECT *  
FROM EMPLOYEES  
AT (  
    TIMESTAMP => '2026-06-05 03:42:32.612 -0700'  
)  
WHERE EMP_ID = 2;
```

Verify:

```
SELECT * FROM EMPLOYEES;
```

Output:

	EMP_ID	EMP_NAME	SALARY
1	1	Alice	50000
2	2	Bob	60000
3	3	Charlie	70000

Alternative Recovery Using OFFSET

Recover table state from 5 minutes ago:

```
SELECT *  
FROM EMPLOYEES  
AT (  
    OFFSET => -60*5  
);
```

Time Travel Use Cases

1. Recover accidentally deleted records.
2. Compare historical and current datasets.
3. Audit data changes.
4. Restore tables after errors.
5. Investigate unexpected data modifications.

Conclusion

Successfully implemented:

1. Real-Time Analytics Pipeline using Streams and Tasks.
2. Dashboard integration with Power BI/Tableau.
3. Historical data analysis using Time Travel.
4. Recovery of deleted records using Time Travel.
5. Automated near real-time reporting architecture in Snowflake.